

===== :

Python Basics

Author: Nagesh , KL University
Hyderabad

Objective: Learn Python fundamentals for
Data Science - Part 1

===== :



```
In [ ]: # =====  
# STEP 1: Hello World  
# =====  
print("Hello, Python!")  
print(10 + 5)
```

```
In [ ]: #=====  
# STEP 2: Variables  
# =====  
a = 10  
b = 20  
print("Value of a:", a)  
print("Value of b:", b)
```

```
In [ ]: name = "KL University"  
print("Name:", name)
```

```
In [ ]: # =====  
# STEP 3: Data Types  
# =====  
x = 10  
y = 10.5  
text = "Python"  
flag = True  
  
print(type(x))  
print(type(y))  
print(type(text))  
print(type(flag))
```

```
In [ ]: # =====  
# STEP 4: User Input  
# =====
```

```
# Uncomment to test
num = int(input("Enter a number: "))
print("You entered:", num)
```

```
In [ ]: # =====
# STEP 5: Conditional Statements
# =====
num = 5

if num > 10:
    print("Greater than 10")
else:
    print("Less than or equal to 10")
```

```
In [ ]: # =====
# STEP 6: Loops
# =====
print("For Loop:")
for i in range(5):
    print(i)
```

```
In [ ]: for i in range(4, 21, 2):
        print(i)
```

```
In [ ]: for i in range(4, 21):
        print(i)
```

```
In [ ]: print("While Loop:")
i = 1
while i <= 5:
    print(i)
    i += 1
```

```
In [ ]:
```

```
In [ ]: # =====
# STEP 7: Lists
# =====
numbers = [10, 20, 30, 40]
print(numbers[3])
```

```
In [ ]: numbers.append(50)
```

```
In [ ]: numbers
```

```
In [ ]: for num in numbers:
        print(num)
```

```
In [ ]: # =====
# STEP 8: Dictionaries
# =====
student = {
    "name": "Hanish",
    "age": 21,
    "marks": 90
}
```

```
In [ ]: print(student["name"],student["age"])
```

```
In [ ]: print(student["name"])
```

```
In [ ]: student["grade"] = "A"
print(student)
```

```
In [ ]: student
```

```
In [ ]: # =====
# STEP 9: Functions
# =====
def sub(a, b):
    return a - b
```

```
In [ ]:
```

```
In [ ]: result = sub(10, 20)
print("Sum:", result)
```

```
In [ ]: # =====
# STEP 10: Programs
# =====
# Even or Odd
num = 7
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

```
In [ ]: # Sum of numbers
total = 0
for i in range(1, 6):
    total += i
print("Sum:", total)
```

```
In [ ]: # Largest number
a = 10
b = 25
c = 15

if a > b and a > c:
    print("A is largest")
elif b > c:
    print("B is largest")
else:
    print("C is largest")
```

```
In [ ]: print("Lab Completed Successfully!")
```

===== :

NUMPY LAB (Python for Data Science - Part 2)

===== :

```
In [ ]: #STEP 1: Import NumPy  
import numpy as np
```

```
In [ ]: # STEP 2: Create Arrays  
arr1 = np.array([1, 2, 3, 4, 5])  
print("Array:", arr1)
```

```
In [ ]: # 2D Array  
arr2 = np.array([[1, 2, 3,4], [4, 5, 6,7]])  
print("2D Array:", arr2)
```

```
In [ ]: # STEP 3: Array Properties  
print("Shape:", arr2.shape)  
print("Dimensions:", arr2.ndim)  
print("Data Type:", arr2.dtype)
```

```
In [ ]: # STEP 4: Special Arrays  
zeros = np.zeros((2,3))  
ones = np.ones((2,3))  
identity = np.eye(4)  
  
print("Zeros:", zeros)  
print("Ones:", ones)  
print("Identity Matrix:", identity)
```

```
In [ ]: # STEP 5: Array Operations  
arr = np.array([10, 20, 30, 40])  
print("Addition:", arr + 5)  
print("Multiplication:", arr * 2)  
print("Square:", arr ** 2)
```

```
In [ ]: # STEP 6: Mathematical Functions  
print("Mean:", np.mean(arr))  
print("Median:", np.median(arr))  
print("Standard Deviation:", np.std(arr))
```

```
In [ ]: # STEP 7: Indexing & Slicing  
print("First element:", arr[0])  
print("Slice:", arr[2:4])
```

```
In [ ]: # STEP 8: Reshaping  
arr3 = np.arange(1, 10)  
reshaped = arr3.reshape(3,3)  
print("Array",arr3)  
print("Reshaped:", reshaped)
```

```
In [ ]: # STEP 9: Random Numbers  
rand = np.random.rand(3,3)
```

```
print("Random:", rand)
```

```
In [ ]: # STEP 10: Aggregation
print("Array",arr)
print("Sum:", np.sum(arr))
print("Max:", np.max(arr))
print("Min:", np.min(arr))
```

```
In [ ]: print("NumPy Lab Completed!")
```

=====

PANDAS LAB (Python for Data Science - Part 3)

=====



```
In [1]: # STEP 1: Import Pandas
import pandas as pd
```

```
In [2]: # STEP 2: Create DataFrame
# From dictionary
student_data = {
    'Name': ['A', 'B', 'C', 'D'],
    'Age': [20, 21, 19, 22],
    'Marks': [85, 90, 78, 92]
}
```

```
In [3]: student_data
```

```
Out[3]: {'Name': ['A', 'B', 'C', 'D'],
        'Age': [20, 21, 19, 22],
        'Marks': [85, 90, 78, 92]}
```

```
In [4]: df = pd.DataFrame(student_data)
```

```
In [5]: df
```

```
Out[5]:
```

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78
3	D	22	92

```
In [8]: # STEP 3: Basic Inspection
#df.head()
```

```
#df.info()
df.describe()
```

```
Out[8]:
```

	Age	Marks
count	4.000000	4.000000
mean	20.500000	86.250000
std	1.290994	6.238322
min	19.000000	78.000000
25%	19.750000	83.250000
50%	20.500000	87.500000
75%	21.250000	90.500000
max	22.000000	92.000000

```
In [9]: df['Marks']
```

```
Out[9]:
```

0	85
1	90
2	78
3	92

Name: Marks, dtype: int64

```
In [10]: df[['Name', 'Marks']]
```

```
Out[10]:
```

	Name	Marks
0	A	85
1	B	90
2	C	78
3	D	92

```
In [12]: df
```

```
Out[12]:
```

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78
3	D	22	92

```
In [14]: #df.iloc[row_index, column_index]
df.iloc[1,2] # selects First row
```

```
Out[14]: 90
```

```
In [15]: df.iloc[[0]]
```

```
Out[15]:
```

	Name	Age	Marks
0	A	20	85

```
In [16]: df.iloc[0:3] #Select Multiple Rows
```

```
Out[16]:
```

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78

```
In [17]: df.iloc[0,1] # intetsection element of first and second rows
```

```
Out[17]: 20
```

```
In [18]: df.iloc[:,1] # selects entire second column
```

```
Out[18]: 0    20
1    21
2    19
3    22
Name: Age, dtype: int64
```

```
In [20]: df.iloc[:,0:2] #Select Entire Multiple Columns
```

```
Out[20]:
```

	Name	Age
0	A	20
1	B	21
2	C	19
3	D	22

```
In [ ]: df
```

```
In [21]: df.iloc[0:2,0:2] # intersection elements of Rows 1st ,2nd and columns 2nd
```

```
Out[21]:
```

	Name	Age
0	A	20
1	B	21

```
In [23]: df
```

```
Out[23]:
```

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78
3	D	22	92

```
In [24]: df.iloc[-1] # Select Last Row
```

```
Out[24]: Name      D
Age      22
Marks    92
Name: 3, dtype: object
```

```
In [25]: df
```

```
Out[25]:
```

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78
3	D	22	92

```
In [30]: new_row = pd.DataFrame({
        'Name': ['D'],
        'Age': [35],
        'Marks': [90]
    })
df = pd.concat([df, new_row], ignore_index=True)
```

```
In [31]: df
```

```
Out[31]:
```

	Name	Age	Marks	Grade
0	A	20	90	B
1	B	21	95	A
2	C	19	83	C
3	D	22	97	A
4	D	35	90	NaN

```
In [26]: # STEP 5: Add / Modify Columns
df['Grade'] = ['B', 'A', 'C', 'A']
df
```


Out[26]:

	Name	Age	Marks	Grade
0	A	20	85	B
1	B	21	90	A
2	C	19	78	C
3	D	22	92	A

In [27]: *# Modify values*
df['Marks'] = df['Marks'] + 5

In [28]: df

Out[28]:

	Name	Age	Marks	Grade
0	A	20	90	B
1	B	21	95	A
2	C	19	83	C
3	D	22	97	A

In []: df.loc[0, 'Marks'] = df.loc[0, 'Marks'] + 5

In []: df

In [34]: *# ♦ STEP 6: Filtering Data*
high_marks = df[df['Marks'] >= df['Marks'].max()]
high_marks

Out[34]:

	Name	Age	Marks	Grade
3	D	22	97	A

In [36]: *# Create sample with missing*
sample = pd.DataFrame({
 'A': [1, 2, None, 4],
 'B': [5, None, 7, 8]
})

In [37]: sample

Out[37]:

	A	B
0	1.0	5.0
1	2.0	NaN
2	NaN	7.0
3	4.0	8.0

In [38]: sample.isnull().sum()

```
Out[38]: A    1
        B    1
        dtype: int64
```

```
In [39]: # Fill missing
sample['A'] = sample['A'].fillna(sample['A'].mean()),0
sample['B'] = sample['B'].fillna(0)

sample
```

```
Out[39]:
```

	A	B
0	1.000000	5.0
1	2.000000	0.0
2	2.333333	7.0
3	4.000000	8.0

```
In [40]: # STEP 9: GroupBy Operation
group = df.groupby('Grade')['Marks'].mean()
print("Average Marks by Grade:", group)
```

```
Average Marks by Grade: Grade
A    96.0
B    90.0
C    83.0
Name: Marks, dtype: float64
```

```
In [43]: # STEP 10: Sorting
sorted_df = df.sort_values(by='Marks', ascending=False)
sorted_df
```

```
Out[43]:
```

	Name	Age	Marks	Grade
3	D	22	97	A
1	B	21	95	A
0	A	20	90	B
4	D	35	90	NaN
2	C	19	83	C

```
In [44]: print("Pandas Lab Completed!")
```

```
Pandas Lab Completed!
```

DATA VISUALIZATION LAB (Part 4)

```
In [ ]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: # Use seaborn dataset for visualization
viz_df = sns.load_dataset('titanic')
```

```
In [ ]: viz_df.head()
```

```
In [ ]: viz_df.isnull().sum()
```

```
In [ ]: # STEP 2: Histogram
plt.figure()
plt.hist(viz_df['age'].dropna())
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

```
In [ ]: # STEP 2: Histogram
plt.figure()
plt.hist(viz_df['age'].dropna(), label="Age Distribution")
plt.title("Age Distribution")
plt.legend()
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

```
In [ ]: # STEP 2: Histogram
plt.figure()
plt.hist(viz_df['age'].dropna(), color='red', label="Age Distribution")
plt.title("Age Distribution")
plt.legend()
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

```
In [ ]: #transparency (opacity)
# 0.1 Completely transparent (invisible)
# 0.5 Semi-transparent
#1 Fully visible (default)

# STEP 2: Histogram
plt.figure()
plt.hist(viz_df['age'].dropna(), color='red', label="Age Distribution", alpha = 0.1)
plt.title("Age Distribution")
```

```
plt.legend()
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

In []: `import matplotlib.pyplot as plt`

```
x = [1, 2, 3, 4, 5]
y = [2, 8, 5, 15, 9]

# Different marker styles
plt.plot(x, y, marker='o') # 'o' → circle
# plt.plot(x, y, marker='s') # 's' → square
# plt.plot(x, y, marker='^') # '^' → triangle up
# plt.plot(x, y, marker='v') # 'v' → triangle down
# plt.plot(x, y, marker='*') # '*' → star
# plt.plot(x, y, marker='x') # 'x' → x mark
# plt.plot(x, y, marker='+') # '+' → plus
# plt.plot(x, y, marker='D') # 'D' → diamond
# plt.plot(x, y, marker='p') # 'p' → pentagon
# plt.plot(x, y, marker='h') # 'h' → hexagon

plt.title("Markers Example")
plt.show()
```

In []: `import matplotlib.pyplot as plt`

In []: `# Line style`
`plt.plot(x, y, linestyle='--')` # dashed line

```
# Available line styles:
# '-' → solid line (default)
# '--' → dashed line
# ':' → dotted line
# '-.' → dash-dot line

# You can try:
# plt.plot(x, y, linestyle='-') # solid
# plt.plot(x, y, linestyle='--') # dashed
# plt.plot(x, y, linestyle=':') # dotted
# plt.plot(x, y, linestyle='-.') # dash-dot

plt.title("Line Style Example")
plt.show()
```

In []: `# Color attribute`
`plt.plot(x, y, color='red')` # red line

```
# Common colors:
# 'blue', 'green', 'red', 'cyan', 'magenta', 'yellow', 'black', 'white'

# Short forms:
# 'b' → blue
# 'g' → green
# 'r' → red
# 'c' → cyan
# 'm' → magenta
# 'y' → yellow
# 'k' → black
```

```
# 'w' → white

plt.title("Color Example")
plt.show()
```

```
In [ ]: # Marker styles
plt.plot(x, y, marker='o') # circle

# Available markers:
# 'o' → circle
# 's' → square
# '^' → triangle up
# 'v' → triangle down
# '<' → triangle left
# '>' → triangle right
# '*' → star
# 'x' → x mark
# '+' → plus
# 'D' → diamond
# 'p' → pentagon
# 'h' → hexagon
# '.' → point
# ',' → pixel

plt.title("Marker Example")
plt.show()
```

```
In [ ]: # Line width
plt.plot(x, y, linewidth=3) # thicker line

# Values:
# linewidth=1 → thin
# linewidth=2 → normal
# linewidth=5 → thick

plt.title("Line Width Example")
plt.show()
```

```
In [ ]: # Marker size
plt.plot(x, y, marker='o', markersize=10)

# Values:
# markersize=5 → small
# markersize=10 → medium
# markersize=15 → large

plt.title("Marker Size Example")
plt.show()
```

```
In [ ]: # Marker face color
plt.plot(x, y, marker='o', markerfacecolor='yellow')

# You can use any color:
# 'red', 'blue', 'green', etc.

plt.title("Marker Face Color")
plt.show()
```

```
In [ ]: # Marker edge color
plt.plot(x, y, marker='o', markeredgewidth='black')

plt.title("Marker Edge Color")
plt.show()
```

```
In [ ]: # Alpha (transparency)
plt.plot(x, y, alpha=0.5)

# Values:
# alpha=0   → invisible
# alpha=0.5 → semi-transparent
# alpha=1   → fully visible

plt.title("Alpha Example")
plt.show()
```

```
In [ ]: # STEP 4: Box Plot
plt.figure()
sns.boxplot(x='survived', y='age', data=viz_df)
plt.title("Age vs Survival")
plt.show()
```

```
In [ ]: # Label and Legend
plt.plot(x, y, label='My Line')

# Legend positions:
# 'upper right', 'upper left', 'lower right', 'lower left', 'center'

plt.legend(loc='upper right')

plt.title("Legend Example")
plt.show()
```

```
In [ ]: # Grid
plt.plot(x, y)

plt.grid(True)    # show grid
# plt.grid(False) → hide grid

plt.title("Grid Example")
plt.show()
```

```
In [ ]: # Axis Limits
plt.plot(x, y)

plt.xlim(0,6)    # x-axis range
plt.ylim(0,20)   # y-axis range

plt.title("Axis Limits")
plt.show()
```

```
In [ ]: import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4, 5]

y1 = [2, 4, 6, 8, 10]    # Line 1 (Linear)
```

```

y2 = [1, 3, 5, 7, 9]      # Line 2
y3 = [2, 5, 3, 7, 4]      # Line 3 (non-linear)

# Plot multiple Lines
plt.plot(x, y1, label='Line 1', color='blue')  # first Line
plt.plot(x, y2, label='Line 2', color='red')   # second Line
plt.plot(x, y3, label='Line 3', color='green') # third Line

# Labels and title
plt.title("Multiple Line Graph")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")

# Legend to identify Lines
plt.legend()

# Grid for clarity
plt.grid(True)

plt.show()

```

```

In [ ]: import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4, 5]

y1 = [2, 4, 6, 8, 10]
y2 = [1, 3, 5, 7, 9]
y3 = [2, 5, 3, 7, 4]

# Plot Line 1 (fully styled)
plt.plot(x, y1,
         color='blue',           # Line color
         linestyle='-',         # solid line
         linewidth=2,           # thickness
         marker='o',             # marker type
         markersize=8,           # marker size
         markerfacecolor='yellow', # inside color
         markeredgecolor='black', # border color
         alpha=0.9,              # transparency
         label='Linear Growth')  # Legend Label

# Plot Line 2
plt.plot(x, y2,
         color='red',
         linestyle='--',
         linewidth=2,
         marker='s',
         markersize=8,
         markerfacecolor='white',
         markeredgecolor='black',
         alpha=0.8,
         label='Odd Pattern')

# Plot Line 3
plt.plot(x, y3,
         color='green',
         linestyle=':',
         linewidth=2,
         marker='^',

```

```

        markersize=8,
        markerfacecolor='pink',
        markeredgecolor='black',
        alpha=0.7,
        label='Non-Linear')

# Title and Labels
plt.title("Complete Styled Multiple Line Graph", fontsize=14)
plt.xlabel("X Axis", fontsize=12)
plt.ylabel("Y Axis", fontsize=12)

# Axis Limits
plt.xlim(0, 6)
plt.ylim(0, 12)

# Grid
plt.grid(True, linestyle='--', alpha=0.5)

# Legend
plt.legend(loc='upper left', fontsize=10)

# Show plot
plt.show()

```

```

In [ ]: import matplotlib.pyplot as plt

plt.figure()

plt.scatter(viz_df['age'], viz_df['fare'])

plt.title("Age vs Fare")
plt.xlabel("Age")
plt.ylabel("Fare")

plt.show()

```

SEABORN

```

In [ ]: import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = sns.load_dataset('titanic')

df.head()

```

```

In [ ]: sns.scatterplot(x='age', y='fare', data=df) # relationship between age and fare
plt.title("Age vs Fare")
plt.show()

```

```

In [ ]: sns.lineplot(x='age', y='fare', data=df) # trend
plt.show()

```

```

In [ ]: import numpy as np
# Python List (array-like)
py_list = [1, 2, 3, 4]

```



```
# NumPy array
np_array = np.array([1, 2, 3, 4])

print(type(py_list))
print(type(np_array))
```

```
In [ ]: sns.barplot(x='sex', y='survived', data=df) # survival rate by gender
plt.title("Survival Rate by Gender")
plt.show()
```

```
In [ ]: sns.countplot(x='survived', data=df) # count of survived vs not
plt.title("Survival Count")
plt.show()
```

```
In [ ]: sns.histplot(df['age'].dropna()) # age distribution
plt.title("Age Distribution")
plt.show()
```

```
In [ ]: sns.kdeplot(df['age'].dropna()) # smooth curve
plt.show()
```

```
In [ ]: sns.boxplot(x='class', y='fare', data=df) # fare distribution by class
plt.show()
```

```
In [ ]: sns.stripplot(x='class', y='age', data=df) # scattered points
plt.show()
```

```
In [ ]: sns.swarmplot(x='class', y='age', data=df) # non-overlapping points
plt.show()
```

```
In [ ]: sns.pairplot(df[['age', 'fare', 'survived']]) # relationships
plt.show()
```

```
In [ ]: corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True, cmap='coolwarm') # correlation matrix
plt.show()
```

```
In [ ]: sns.jointplot(x='age', y='fare', data=df)
plt.show()
```

```
In [ ]: sns.regplot(x='age', y='fare', data=df) #REGRESSION PLOT
plt.show()
```

```
In [ ]: sns.residplot(x='age', y='fare', data=df) # RESIDUAL PLOT
plt.show()
```

```
In [ ]: sns.countplot(x='survived', hue='sex', data=df) # male vs female
plt.title("Survival by Gender")
plt.show()
```

```
In [ ]: sns.countplot(x='survived', hue='sex', data=df, palette='Set2')

# Common palettes:
# 'deep', 'muted', 'bright', 'pastel', 'dark', 'colorblind'
```

```
# 'Set1', 'Set2', 'Set3', 'coolwarm', 'viridis'

plt.show()
```

```
In [ ]: sns.set_style("whitegrid")    # grid background

# Other styles:
# 'darkgrid', 'whitegrid', 'dark', 'white', 'ticks'

sns.countplot(x='survived', hue='sex', data=df)
plt.show()
```

```
In [ ]: sns.set_style("darkgrid")

sns.histplot(df['age'].dropna(), kde=True, color='green')

plt.title("Age Distribution")
plt.show()
```

```
In [ ]: sns.barplot(x='class', y='survived', hue='sex',
                    data=df, palette='coolwarm')

plt.title("Survival by Class and Gender")
plt.show()
```

```
In [ ]: # Python List
print(py_list * 2)

# NumPy array
print(np_array * 2)
```

```
In [ ]: # Python List
print(py_list + py_list)

# NumPy array
print(np_array + np_array)
```

```
In [ ]: # Python List (nested)
py_list_2d = [[1, 2], [3, 4]]

# NumPy array
np_array_2d = np.array([[1, 2], [3, 4]])

print(np_array_2d.shape)
#out put (2, 2)
print(py_list_2d.shape)
#AttributeError: 'list' object has no attribute 'shape'
```

```
In [ ]: py_list_2d
```

```
In [ ]: # NumPy is faster for large data
# (No need to write Loop)
```

```
np_array = np_array + 10
print(np_array)
```

```
In [ ]: print(py_list[0])      # 1
        print(np_array[0])    # 1
```

```
In [ ]: list1 = [20,30,40,50]
        list1[0] = list1[0] + 200
```

```
In [ ]: list1
```

```
In [ ]: import numpy as np
        arr = np.array([1,2,3,5])
```

```
In [ ]: arr[1] = arr[1] + 20
```

```
In [ ]: print(arr)
```

```
In [ ]: import numpy as np

# Python List
list1 = [20, 30, 40, 50]

# NumPy array
arr = np.array([20, 30, 40, 50])

# ♦ Update single value
list1[1] = 100
arr[1] = 100

print("List:", list1)
print("Array:", arr)

# ♦ Update multiple values
list1[1:3] = [200, 300]
arr[1:3] = [200, 300]

print("List:", list1)
print("Array:", arr)

# ♦ Conditional update
# List (needs loop)
for i in range(len(list1)):
    if list1[i] == 40:
        list1[i] = 400

# NumPy (direct)
arr[arr == 40] = 400

print("List:", list1)
print("Array:", arr)
```

```
In [1]: # =====
        # 1. Python automatic datatype
        # =====
```

```

x = 10          # Python assigns int automatically
y = 10.5        # Python assigns float automatically
z = "Hello"     # Python assigns string

print(type(x))  # int
print(type(y))  # float
print(type(z))  # str

# =====
# 2. NumPy automatic datatype
# =====

import numpy as np

a = np.array([1, 2, 3]) # NumPy assigns datatype automatically
print(a.dtype)          # usually int64

# =====
# 3. NumPy manual datatype
# =====

b = np.array([1, 2, 3], dtype=np.int32) # force datatype
print(b.dtype)                          # int32

# =====
# 4. Large number → automatic int64
# =====

c = np.array([10000000000]) # Large number
print(c.dtype)              # int64 (needs more memory)

# =====
# 5. Pandas automatic datatype
# =====

import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],          # numeric → int64
    'B': [10.5, 20.2, 30.1], # float → float64
    'C': ['x', 'y', 'z']     # text → object
})

print(df.dtypes)

# =====
# 6. Pandas manual datatype change
# =====

df['A'] = df['A'].astype('int32') # convert int64 → int32
print(df.dtypes)

# =====
# 7. Memory difference

```

```
# =====  
  
arr1 = np.array([1,2,3], dtype=np.int32)  
arr2 = np.array([1,2,3], dtype=np.int64)  
  
print(arr1.nbytes) # Less memory  
print(arr2.nbytes) # more memory
```

```
<class 'int'>  
<class 'float'>  
<class 'str'>  
int32  
int32  
int64  
A      int64  
B      float64  
C      object  
dtype: object  
A      int32  
B      float64  
C      object  
dtype: object  
12  
24
```

In []: